Alzheimer's Disease Prediction Using Modern Machine Learning Techniques

A Project report

submitted in partial fulfillment of the requirements for the award of the degree

MASTER OF COMPUTER APPLICATIONS

Submitted by

VADLAMUDI SRINIVASULU 23MD1F0067

Under the esteemed guidance of

Mr.K.Lakshmi Narayana, MCA.

Associate Professor

DEPARTMENT OF COMPUTER APPLICATIONS

RAJAMAHENDRI INSTITUTE OF ENGINEERING AND TECHNOLOGY

(APPROVED BY AICTE, AFFILIATED TO JNTUK, KAKINADA, ACCEDITED TO NAAC)

BHOOPALATNAM, RAJAMAHENDRAVARAM – 533107

MASTER OF COMPUTER APPLICATIONS

2023 – 2025

RAJAMAHENDRI INSTITUTE OF ENGINEERING&TECHNOLOGY

(Approvedby AICTE, Affiliatedto JNTUK, Kakinada and Accreditedto NAAC)

BHOOPALATNAM, RAJAMAHENDRAVARAM – 533107

DEPARTMENT OF COMPUTER APPLICATIOMS

CERTIFICATE

This is to certify that V.SRINIVASULU– 23MD1F0067of second year MCA., had carried out the main project work on ***"Alzheimer's Disease Prediction Using Modern Machine Learning Techniques"****, for the Partial fulfillment of the award of the degree* ***Master of Computer Applications*** *(MCA) in* ***Rajamahendri Institute of Engineering & Technology****, RAJAMAHENDRAVARAM (Affiliated to JNTU, Kakinada) is a Bonafide record of the work done by them during the academic year 2023-2025 under the guidance and supervision. The results of this project have not been submitted to any other University or Institute for the award of any degree.*

Internal Guide	Head of the Department
Mr. K.Lakshmi Narayana, MCA	**CH. SuryaPrakash,MCA**
Associate Professor.	*Associate Professor.*

External Examiner

DISSERTATION CERTIFICATE

This is to certify that the dissertation entitle **"Alzheimer's Disease Prediction Using Modern Machine Learning Techniques"** *by* MR. VADLAMUDISRINIVASULU (23MD1F0067) student of **Master of Computer Applications (MCA)** *in***Rajamahendri Institute of Engineering & Technology**, *Rajamahendravaram, Permanently affiliated to***Jawaharlal Nehru Technological University, Kakinada**,*is here by accepted and approved as a credible work. It is further satisfied that this work has not been submitted for similar purpose anywhere else. His work has been found satisfactory for the partial fulfilment of the award of the degree of* **MCA.**

INTERNAL EXAMINER ***EXTERNAL EXAMINER***

Head of the Department

Department of Computer Applications

Rajamahendri Institute Of Engineering & Technology

STUDENTS DECLARATION

We here by declare that the project report entitled **"Alzheimer's Disease Prediction Using Modern Machine Learning Techniques"***submitted by us to* **Rajamahendri Institute of Engineering and Technology, Rajahmundry** *in partial fulfillmentof the requirement for the award of the degree of Master's in* **COMPUTER APPLICATIONS** *is a record of bonafide project work carried out by us under the guidance of* **Mr.K.Lakshmi Narayana.***We further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.*

Place: Rajahmundry

Date:

(Signature)

V.Srinivasulu

Regd.No: **23MD1F0067**

ACKNOWLEDGMENT

We are profoundly grateful to express our gratitude and respect towards our Internal Guide **MR.K.LAKSHMI NARAYANA** and Head of the Department **MR. CH. SURYA PRAKASH**, for their excellent guidance right from the selection of projects and valuable suggestions throughout the project work. We are thankful to them for giving this opportunity to work in the laboratory at any time. Their constant encouragement and support have been the cause for success, in completing this in college. They have given tremendous support, both on the technical and moral front.

We also thank the **Project Review Committee** and all **Faculty Members of MCA Department**, for their valuable suggestions and for guiding us throughout the project and leading us in completing of project efficiently.

We would like to thank all the members of the teaching and non-teaching staff for all their support in completion of our project.

We are thankful to **DR. K. BHASKAR MUTYALU, PRINCIPAL**, for his support during and till the completion of the project.

We extend our thanks to college Management for their support and encouragement for the project.

We heartfully thank our **parents** and **well-wishers** for their support and encouragement for the successful completion of the project.

V.SRINIVASULU

23MD1F0067

ABSTRACT

Because of its revolutionary impact on the scientific community, ML (Machine Learning) has become the most prominent application of AI. The purpose of this work is to apply machine learning to the problem of diagnosing Alzheimer's disease. There are a lot of individuals impacted by Alzheimer's disease, and the illness takes lives all over the world. Utilizing machine learning methods, one may ascertain if an individual suffers from Alzheimer's disease by factoring in symptoms such as chest pain, high cholesterol, age, etc. In this paper, four supervised ML algorithms are utilized: K-Nearest Neighbour (90%), Random Forest (89%), Artificial Neural Networks (87%), and Logistic Regression (87%). The accuracy of the predictions made by each of these algorithms ranges from 87% to 90%.

Keywords — Machine Learning, KNN, ANN, Random Forest, Logistic Regression, Alzheimer's, Disease Prediction, Machine Learning in Healthcare

.

TABLES OF CONTENTS

INDEX **Page No**

1.INRODUCTION **1**

2. SYSTEM ANALYSIS **2**

2.1. EXISTING SYSTEM 2

2.1.1. DISADVANTAGES OF EXISTING SYSTEM 2

2.2. PROPOSED SYSTEM 2

2.1.1. ADVANTAGES OF PROPOSED SYSTEM 3

2.3. FEASIBILITY STUDY 3

2.3.1. DETERMINING THE PROFITABILITY 3

2.3.2.DEVELOPMENTAL LIGIGANCE 3

2.4. SOFTWARE REQUIREMENTS & SPECIFICATIONS 4

2.4.1. SOFTWARE REQUIREMENTS 4

2.4.2. HARDWARE REQUIREMENTS 4

2.5. SOFTWARE DEVELOPMENT LIFE CYCLE 4

2.5.1. HYBRID &MULTI-MODEL MODELS 6

3. SYSTEM DESIGN7

3.1. DATA FLOW DIAGRAM 7

3.1.1. INPUT DESIGN 8

3.2.2. OUTPUT DESIGN 9

3.2. ALGORITHMS 10

3.3. UML DIAGRAMS 9

3.3.1. GOALS 10

3.3.2. USECASE DIAGRAM 10

3.3.3. CLASS DIAGRAM 11

3.3.4. SEQUENCE DIAGRAM 11

3.3.5. ACTIVITY DIAGRAM 12

4. IMPLEMENTATION 14

4.1. MODULES 14

4.1.1. MODULES DESCRIPTION 14

4.2. PYTHON 15

4.3. DJANGO 22

4.4. SOURCE CODE 27

5.SYSTEM TESTING 37

5.1. UNIT TESTING 37

5.1.1. TEST OBJECTIVES 37

5.1.2. FEATURES TO BE TESTED 37

5.2. INTEGRATION TESTING 37

5.3. SYSTEM TESTING 37

5.4. WHITE BOX TESTING 37

5.5. BLACK BOX TESTING 37

5.6. ACCEPTENCE TESTING 38

5.7. SAMPLE TEST CASES 38

6. OUTPUT SCREENS 39

7. CONCLUSION 43

8. FUTURE ENHANCEMENT 44

9. BIBLIOGRAPHY 45

LIST OF FIGURES

S. No.	FIGURE No.	NAME OF THE FIGURE	PAGE No.
1.	2.1	Software Development Life Cycle	6
2.	3.1	Data Flow Diagram	7
3.	3.2	Use case Diagram	10
4.	3.3	Class Diagram	11
5.	3.4	Sequence Diagram	12
6.	3.5	Activity Diagram	13
7.	4.1	Django Framework	23
8.	4.2	Django Architecture	23
9.	6.1- 6.10	Output Screens	39-42

LIST OF TABLES

S. No.	FIGURE No.	NAME OF THE TABLE	PAGE No.
1.	5.1	Sample Test Case	38

1. INTRODUCTION

Predictions based on historical data are the focus of machine learning. One subfield of artificial intelligence, machine learning models the way humans learn and utilizes algorithms to gradually increase a system's accuracy. The algorithm for machine learning consists of two steps: Priorities need to be training and testing. For decades, machine learning software has struggled to diagnose a patient's illness based on their history and symptoms alone. Healthcare issues may be efficiently addressed with the use of machine learning technologies. Utilizing the whole breadth of machine learning, we are monitoring patients' vitals. We can create models that quickly clean, analyze, and provide results from data with the use of machine learning models. With this technology, doctors will be able to make better diagnoses, which means patients will have better information to make treatment choices and healthcare services will be of higher level. Within the framework of healthcare, ML may be most effectively used to the medical field. Using structured data, this study aims to enhance the precision of massive datasets. The present model will use decision trees, KNN, and linear algorithms for illness prediction. The present method is able to predict the occurrence of illnesses that are unique to a particular region and population. Unfortunately, this method is limited to predicting certain illnesses. To predict the likelihood of illnesses, this system employs CNN algorithms and Big Data. Machine learning techniques K-NN, Decision Tree, and Naive Bayesian are used by the system while dealing with S type data.

2. SYSTEM ANALYSIS

2.1Existing System :

The current methods and techniques used to forecast the onset of diseases, such as Alzheimer's and others, have inherent limitations. It is possible that these systems can't manage complicated, high-dimensional medical information since they rely so much on certain algorithms like Naive Bayes, Decision Trees, and K-Nearest Neighbors. The lack of generalizability to other patient populations is another consequence of research concentrating on illnesses that affect particular regions or populations. Medical professionals have a hard time trusting the findings and explaining them to patients because to the lack of interpretability and transparency in the forecasts, which further complicates practical implementation. Inadequate training data is another common problem with current systems, which results in less-than-ideal accuracy and decreased dependability in practical settings.

2.1.1Disadvantages of Existing System:

1. Limited scope in predicting a wide range of diseases due to algorithmic constraints and dataset limitations.
2. Poor generalizability across diverse populations and medical scenarios.
3. Challenges with model interpretability, making it difficult for healthcare professionals to validate and trust predictions.

2.2 Proposed System :

To provide a more thorough and precise method for illness prediction, the suggested system makes use of state-of-the-art machine learning methods including Logistic Regression, Random Forest, and Artificial Neural Networks (ANN). The system maintains its exceptional speed and flexibility by making use of more, better datasets and fine-tuning parameters like tree depth, epoch values, and iterations. By combining various demographic and clinical data, this method facilitates early diagnosis and individualised treatment programs, with an emphasis on individualized healthcare. To further guarantee its efficacy across different demographics and boost confidence

among healthcare providers, the system also includes strong testing and validation procedures.

2.2.1Advantages of Proposed System:

1. Improved accuracy and reliability in disease prediction, supported by advanced algorithms and optimized parameters.
2. Broader applicability across diverse populations and medical conditions, enhancing the system's versatility.
3. Increased transparency and interpretability of predictions, fostering trust and ease of adoption in clinical practice.

2.3FEASIBILITY STUDY

During this phase, we determine if the project is feasible and provide a business proposal that covers the project's main points and approximate cost. In order to determine if the proposed system is feasible, it must undergo system analysis. Our first concern is ensuring that the proposed solution will not put an undue financial burden on the company. Before beginning a feasibility study, it is important to have a general idea of what the system woul d need. The feasibility study takesthree main factors into account:

• The plan's financial viability

•Practicality in terms of technology and society

2.3.1 DETERMINING THE PROFITABILITY

Examining the system's prospective monetary impact on the organization is the driving force behind this study. There is a limited budget for the system's research and development by the company. The expenditure must have a justification. The end result was a system that was both affordable and made extensive use of publicly available technology. It was essential to purchase just the customized goods.

2.3.2 DEVELOPMENTAL LIGIGANCE

In order to ensure that the system meets all of its technical criteria, this research is conducted to assess its technical feasibility. The existing technological resources should not be overloaded by any system that is built. The current technological resources will be put to heavy use as a result of this. The customer will be subject to high expectations as a result of this. Since there would be little to no modification needed to deploy the designed system, its requirements should be low.

Social feasibility (2.3.3)

Finding out how happy the user is with the system is the main goal of the research. This includes instructions on how to maximize the system's potential. Instead of seeing the system as a threat, the user should view it as a necessary tool. Effective methods of familiarizing and educating users with the system have a direct correlation to the level of acceptability among users. Improving his self-assurance will allow him to provide more useful input as the system's end user.

2.4SYSTEMREQUIREMENTS:

2.4.1HARDWARE REQUIREMENTS:

- **System** : Intel i7
- **Hard Disk** :1 TB.
- **Monitor** : 14' Colour Monitor.
- **Mouse** : Optical Mouse.
- **Ram** :8GB.

2.4.2SOFTWARE REQUIREMENTS:

- **Operating system** : Windows 10.
- **Coding Language** : Python.
- **Front-End** : Html. CSS
- **Designing** :Html,css,javascript.
- **Data Base** : SQLite.

2.5 SOFTWARE DEVELOPMENT LIFE CYCLE

CRISP-DM for Alzheimer's Disease Prediction (Customized Flow)

1. Business Understanding

Define problem: Predict Alzheimer's onset early

Stakeholders: Clinicians, neurologists, healthcare providers

2. Data Understanding

Data sources: MRI scans, PET scans, clinical notes, cognitive test scores, .genetic data (like APOE4), etc.

Explore data: Visualizations, correlation analysis, missing data patterns

Data Preparation

Preprocessing: Handle missing data, normalization

Feature selection: Imaging biomarkers, demographics

Data splitting: Train/Test/Validation sets

Modeling

Algorithms: CNN (for MRI), SVM, Random Forest, LSTM (for time-series)

Model tuning: Cross-validation, hyperparameter optimization

Evaluation

Metrics: Accuracy, Precision, Recall, ROC-AUC, F1-score

Clinical relevance: Model interpretability, false negative rate

Deployment

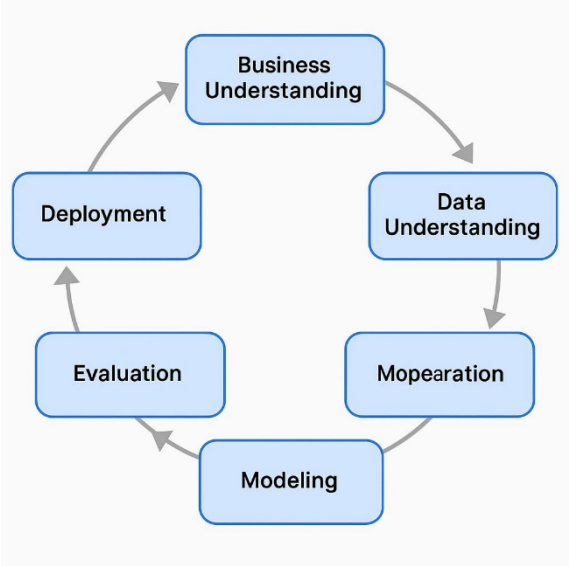
Integration into clinical tools

Model APIs or dashboards

Monitor for performance drift and re-training

Diagram: CRISP-DM Applied to Alzheimer’s PredictionI’ll now generate a diagram showing this CRISP-DM pipeline in the context of Alzheimer’s prediction.CRISP-DM for Alzheimer's Disease Prediction (CustomizedFlow)

DIAGRAM:



2.5.1Hybrid & Multi-Modal Models

a. Multi-Input Deep Learning Models

Combine CNN for imaging + MLP for tabular data (like MMSE scores, APOE status)

Sometimes include attention mechanisms for interpretability

b. Graph Neural Networks (GNNs)

Used for: Brain connectivity data (functional MRI, diffusion tensor imaging).

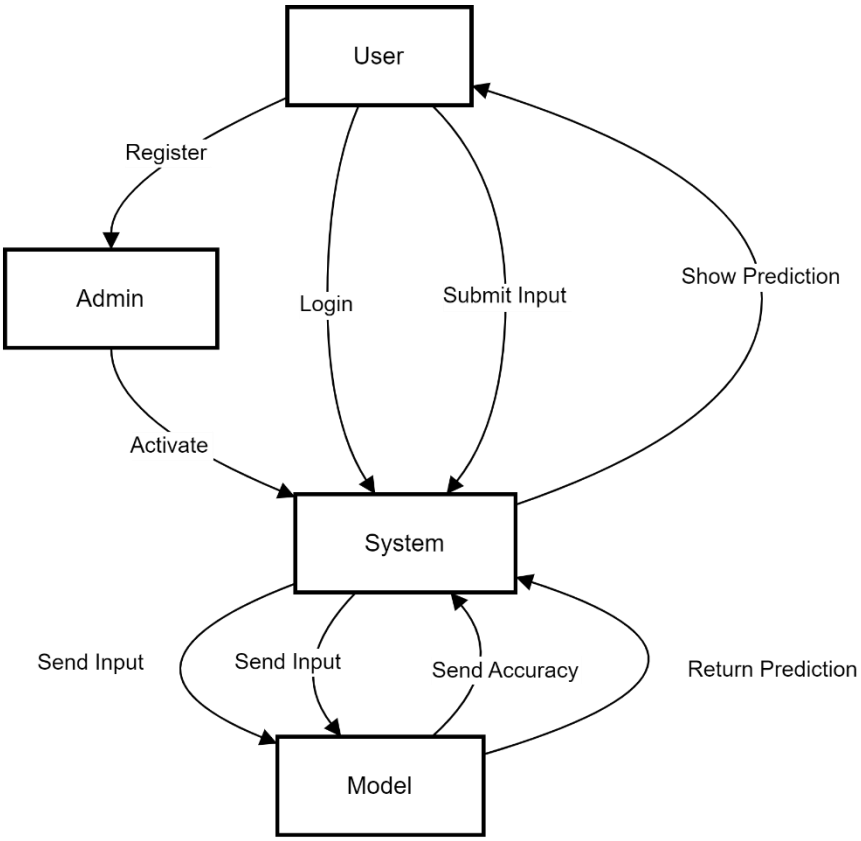
3. SYSTEM DESIGN

3.1 DATA FLOW DIAGRAM:

1. A bubble chart is another name for the DFD. A system may be represented using this basic graphical formalism by first taking data input from the user, then processing that data in different ways, and finally producing data as an output.
2. A crucial tool for modeling is the data flow diagram (DFD). In order to simulate the parts of the system, it is used. Included in this category are the system's process, the data used by it, any external entities involved in the system's interactions, and the information flows inside the system.

Third, DFD reveals the information's transformational journey through the system. It is a visual method for representing the changes that take place when data goes from input to output and how that data flows.

The fourth name for DFD is a bubble chart. Any degree of abstraction may be represented by a DFD. Depending on the amount of information flow and functional intricacy, DFD may be divided into tiers.



3.1.1INPUT DESIGN

Through the input design, the user is linked to the information system. In order to handle raw transaction data, it is necessary to turn it into a usable format, which involves developing data preparation requirements and methods. Both keying in the data by hand and scanning it from a printed page into the computer are viable options for this task. Key objectives of input design include minimizing input, reducing errors, avoiding delays, doing away with superfluous steps, and keeping the process simple. The input has been carefully crafted to provide privacy, ease of use, and security. What data has to be input?¬The following considerations were made by Input Design: What is the best way to code or arrange the information?

- An open forum where operational workers may voice their opinions. - A Guidelines on how to construct input validations and what to do if anything goes wrong.

Initial Objectives

To begin, a user-centric description of the input is the starting point for building a computer system via input design. Helping managers find correct data in the system and avoiding human error during data input are two of the most important functions of thisarchitecture.
2. Simplified data entry interfaces that facilitate user navigation of large data sets are the means by which this is achieved. Input design's holy grail is efficient and error-free data entering. All data adjustments may be done utilizing the data input screen. The opportunity to review records is also provided.
As you enter the data, it will check for accuracy. When inputting data, screens are helpful. Sending out appropriate messages when needed keeps the user from becoming lost in instant pleasure. Thus, the objective of input design is to construct an input arrangement that is easy for users to use.

3.1.2 BOOK COVER DESIGN

When it satisfies the needs of the customer and is simple to grasp, we may say that the final product is of high quality. When a system processes data and then sends the processed results to other systems or people, that's called an output. The replacement of data for both real-time consumption and physical copies should be carefully considered when planning output. For the user, it is the primary and paramount source of information. Intelligent and efficient output design improves the user-system relationship and aids in decision-making.
1. Computer output design requires a methodical and well-planned approach. The output has to be created in a manner that consumers will find straightforward and effective, and it also has to provide the right output. While evaluating the output from the design computer, it is crucial to identify the exact output needed to meet the criterion.
The second step is to choose a data presentation method.
Third, save the results of the system's operations as a report, document, or other kind of document.
The information system's output should accomplish at least one of the following.

Explain in fully what has been done, what is happening now, or what will happen in the future.
• Warn of important events, opportunities, problems, or warnings.
• Cause a domino effect. Ensure a plan of action is correct.

3.2 ALGORITHM

Deep Learning Algorithms (for imaging and time-series data)

a. CNN (Convolutional Neural Networks)

Use: Analyzing brain scans (MRI, PET)

Why: Excellent for spatial data and feature extraction

Example: Classifying Alzheimer's vs. Mild Cognitive Impairment (MCI) from 3D MRI data

b. RNN / LSTM (Recurrent Neural Networks / Long Short-Term Memory)

Use: Time-series data like cognitive scores over time

Why: Captures temporal patterns in disease progression

c. Autoencoders / Variational Autoencoders

Use: Dimensionality reduction, anomaly detection

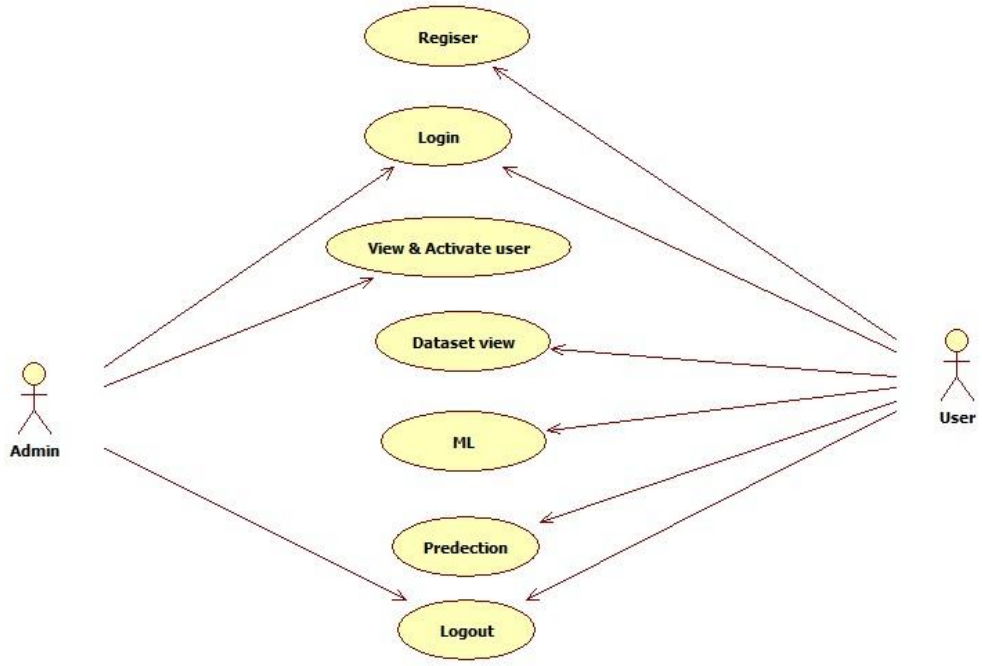
Why: Unsupervised feature learning, especially useful in MRI and multi-

3.3UML DIAGRAMS

United Modeling Language is shortened to UML. One such standard language for object-oriented software engineering is Universal Modeling Language (UML). Both the creation and maintenance of the standard are the purview of the Object Management Group.
One of UML's ultimate aims is to become the de facto language for describing OO programs. The notation and the meta-model are the two main parts of the current UML version. Further, further processes or approaches may be added to or coupled to UML in the future. Software and non-software system artifacts, including business models, may be described, displayed, constructed, and documented using the Unified Modeling Language.

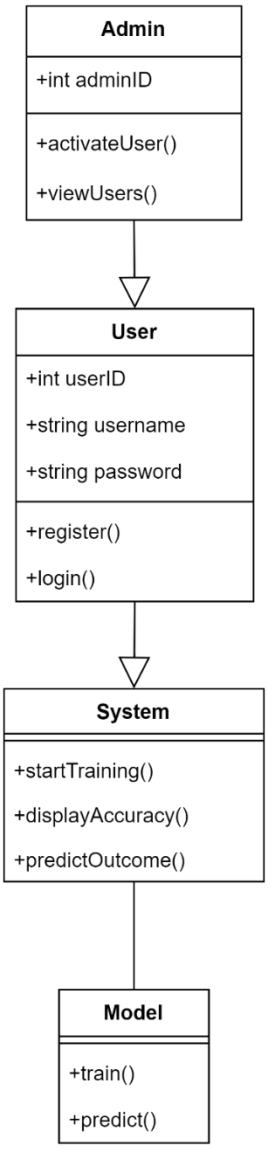
To depict big, complicated systems, the Unified Modeling Language (UML) is the technical gold standard.
Developers working with objects-oriented programming use the Unified Modeling Language (UML) as an essential tool. Visual notations play a significant role in the Unified Modeling Language (UML), a software engineering framework. paragraph 3.3.1.1 Final Product: The primary goals that influenced the creation of the UML are as follows:
1. Provide a ready-to-use expressive visual modeling language so that users may build and share meaningful models.
2. Incorporate tools for specialization and extension to make the core concepts more adaptable.
#3: Don't become too attached to any one programming language or development methodology.
4. Develop a structured approach to understanding modeling terminology.
5. Advocate for the growth of the OO tool market.
Sixth, support more complex development concepts, such as frameworks, components, joint efforts, and patterns.
7. Utilize norms established by the business.

Case diagram 3.3.2: The Unified Modeling Language (UML) defines and generates use-case diagrams, which are a kind of behavioral diagram. The objectives, actors, and interactions between use cases may be better understood with the use of use case diagrams, which are visual representations of the system's operation. A use case diagram's primary goal is to illustrate the various actors and their interactions with the system. It is possible to display the functions performed by each part of the system.



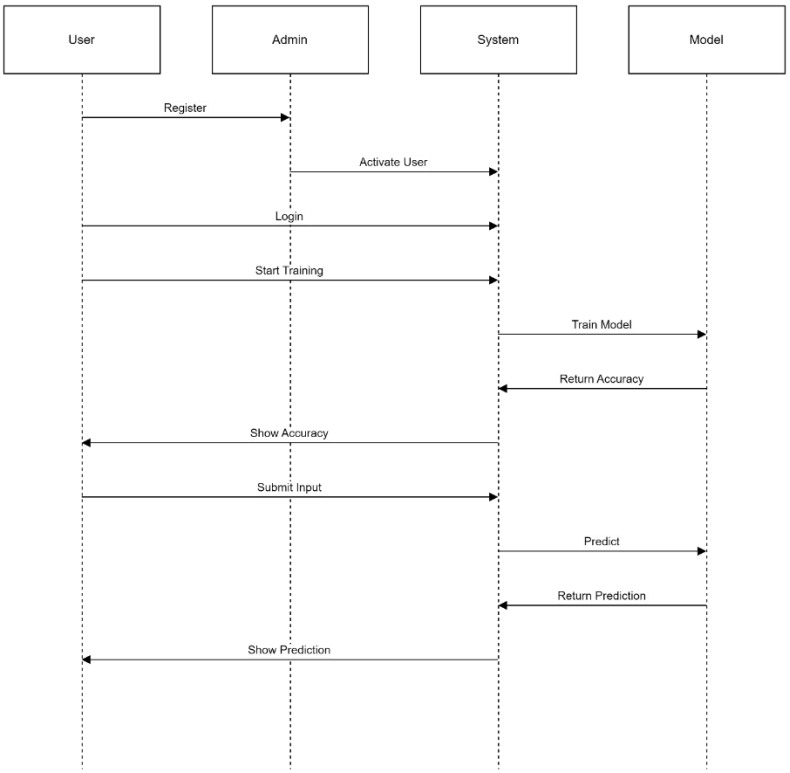
3.3.3 CLASS DIAGRAM:

An example of a static structural diagram used in software engineering is the Unified Modeling Language (UML) class diagram. This diagram shows the classes of a system along with their properties, methods, and connections. That class's information is outlined in it.



3.3.4 SEQUENCE DIAGRAM:

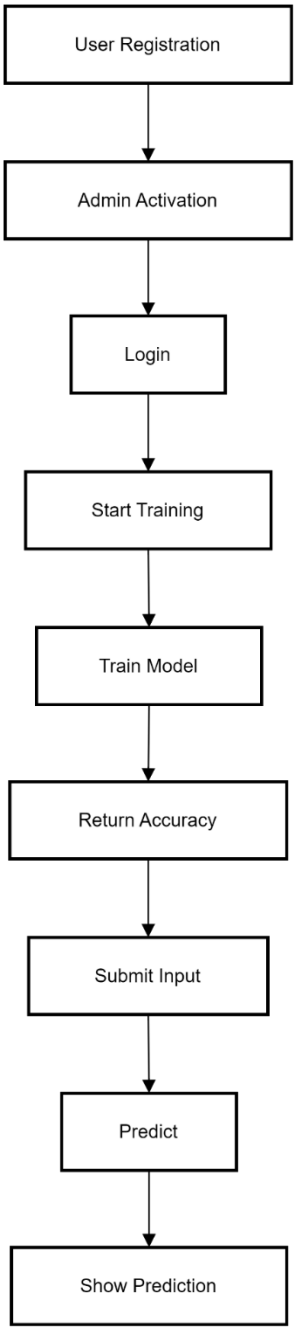
Unified Modeling Language (UML) sequence diagrams illustrate the interdependencies and sequential execution of processes. It is a Message Sequence Chart construction. Alternate names for sequence diagrams include event scenarios, timing diagrams, and event diagrams.



3.3.5 ACTIVITY DIAGRAM:

Activity diagrams make it easy to see how several steps in a process interact with one another, and they also allow features like choice, iteration, and concurrency. Unified Modeling Language activity diagrams may depict the operational and business processes of a system. An activity diagram is one tool for understanding control flow in its broadest sense.

.


User Registration
Admin Activation
Login
Start Training
Train Model
Return Accuracy
Submit Input
Predict
Show Prediction

4. IMPLEMENTATION

4.1 MODULES:

- **User**
- **Admin**
- **Machine Learning**

4.1.1 MODULES DESCRIPTION:

User:

Registration is required of the User. He needed to provide a working email and phone number when he signed up so he could get future messages. The administrator will be able to activate the user when they register. The user will be able to access our system after the administrator has activated them. Users are able to submit datasets that match our dataset columns. Integer or float data types are required for algorithm execution. To conduct our tests, we used the dataset owned by Adacel Technologies Limited. Our Django application also allows users to update old datasets with new data. The data cleaning procedure may be initiated by the user by clicking the Data Preparations button on the website. You will see the cleaned data together with the necessary graph.

Admin:

Login credentials for the administrator are available. The registered users may be activated by the admin. Only the user will be able to access our system after he activates it. In the admin panel, users may be seen, and data can be loaded in the browser for total data viewing. The training data list and the test data list are both viewable by the admin. The administrator may access the data and see the predictions.

Machine learning:

Based on the split criterion, the cleaned data is split into 80% training and 20% test, then the dataset is subjected to one machine learning classifier such as Natural Language Process(NLP). Sentiment analysis by fine tuning auto encoding models like BERT and ALBERT to achieve a comprehensive understanding of public sentiment.

Thus, we have analyzed the results of our experiment and methodology using the contextual information and verified the insights.

4.2 PYTHON

One example of a high-level programming language is Python, which is interpretable, interactive, and designed for general-purpose use. Python is an interpreted language that prioritizes readable and understandable code. Thanks to its syntax, it's feasible to convey concepts with less lines of code than languages like C++ or Java. It uses whitespace indentation to separate pieces of code instead of keywords or curly brackets. Its offered building elements allow for the development of understandable code of any scale. You may find Python interpreters on a wide variety of OSes. All versions of Python, including the most popular one, CPython, are open source software and are developed by a community. For charitable purposes, CPython is overseen by the Python Software Foundation. Python stands out due to two features: its automated memory management and its dynamic type system. The vast standard library that comes with it makes it compatible with numerous programming paradigms, including imperative, procedural, and object-oriented.
Mode of the User Interface Program code
When using the interpreter without passing a script file as an argument, the following prompt will show −

At 13:34:43 on November 11, 2010, this is Python 2.4.3, version 1.1.1. Red Hat 4.1.2-48 is running on Linux2, and GCC 4.1.2 is dated 20080704. Punch in "help," "copyright," "credits," or "license" to find more details. >>> To have Python say "Hello, Python!" when you type the following into the terminal, press the Enter key.
Modern Python implementations of the print statement—as in print ("Hello, Python!");—require the use of parentheses. But this produces the following result in Python 2.4.3. −
Farewell, Python.
Configuration for Code Program code
Invoking the interpreter with a script parameter initiates script execution, which continues until the script is finished. Once filming is over, the interpreter's duties will stop.
Now, let's write a script to construct a simple Python application. For files created in Python, the extension is.py. Create a file named test.py and paste the following code into it.
Give It a Go This is where you say "Hello, Python!"
It is recommended that you include Python in your PATH environment variable by default. Here are the instructions to run this software:
executed the Python script "random test.py"
The following result follows from this →

Farewell, Python.
We could look at other ways of executing the Python script. You can locate the revised

test.py file here. Try It Out Now Press the "Hello, Python!" button to launch Python. The Python interpreter is assumed to be located in the /usr/bin directory in this case. Here are the instructions to run this software:
run the command "chmod +x test.py" # Executability of the file $./test.py will be achieved by doing this.
The following result follows from this →
Farewell, Python.

Python Identifiers

Python objects such as variables, functions, classes, and modules are referred to by their identifiers. A string of alphanumeric characters, an underscore (_), and a string of zero or more letters, underscores, or numbers (0–9) make up an identifier. Python IDs cannot include punctuation characters like @, $, and %. The computer language Python is case sensitive. So, in Python, manpower and manpower are not the same thing.
Python identifiers should be named according to these standards −

All class names begin with a capital letter. A lowercase letter is the starting point for all other identifiers.
An identifier is considered private if it begins with only one underscore. A very confidential identity is one that begins with two leading underscores. The identifier is a special name specified by the language if it finishes with two trailing underscores.
Protected Terms
The Python keywords are shown in the following list. You can't use these terms as variables, constants, or any other kind of identifier since they're reserved. Every single Python keyword is comprised entirely of lowercase characters.
and the head not make a final declaration or take a leave of absence
In the event that the print statement returns del import, attempt elif in; otherwise, deal with the exception of lambda. yield
Indentation and Lines
Python does not come with brackets to denote sections of code for flow control or class and function declarations. The strict use of line indentation to designate blocks of code is mandatory.
The indentation amount may be changed, but it must be uniform across all statements in the block. As an example, if the condition is true, then print "True"; else, print "False".
Nonetheless, an error is produced by the following block: print "Answer" if True print "True" otherwise print "False"
So, in Python, a block would be formed by all the continuous lines that are indented the same amount of spaces. There are many statement blocks in the given example. Important − Do not attempt to comprehend the reasoning just now. Even without braces, be sure you understand different blocks.
Import "#!/usr/bin/python". system (
try:
if (IOError): print "There was an error writing to", file_name; else, open the stream of files using the given format (file_name, "w").
system.exit(); print "Enter '", file_finish,

print "'When finished" but before file_finish:
the value of file_text is set to raw_input("Enter text: "),
If file_text is equal to file_finish, then: # finish editing the file. finish processing the file. execute code snippet into file. put "\n" into the file. Close(); then, in the raw input field, type "Enter filename: ".
checks whether the length of the file name is equal to zero:
copy "Next time please enter something"
attempt: file = open; sys.exit() for "r" in file_name
outside of IOError:
display "There was an error reading file"
print file_text after system exit, file_text = file.read(), file_close() Multiple-Line Proformas
A new line is usually appended at the conclusion of a Python statement. To indicate that the line should continue, Python does indeed permit the use of the line continuation character (\). Take this example: the sum is equal to item one plus item two plus item three.
The line continuation character is not necessary for statements that are enclosed in brackets, curly brackets, or parentheses. To illustrate −

days are a set that contains the days of the week, ordered as follows: Monday through Friday.
Python Blockquote
Python allows string literals to be denoted by single (′), double (″), or triple (‴ or ″″″) quotes, provided that the same kind of quotation is used at both the beginning and the end of the string.
The string is spanned over numerous lines using the triple quotes. One example is that everything listed below is completely within the law: −
word = 'word' sentence = "This is a sentence."
sentence = """This sentence is a paragraph. A number of lines and phrases make it up.
Python comments
Any string literal that does not include a hash symbol (#) indicates the beginning of a remark. The Python interpreter considers all characters after the # symbol and up to the end of the physical line to be part of the remark. Check Out This Live Demo! #!/usr/bin/python Output: "Hello, Python!" initial comment
This leads to the following outcome −
So long, Python!
Name = "Madisetti" # A remark may be typed on the line immediately after a statement or expression. Once again, this is a remark
Multiple lines may be commented in the following way −

This is a remark.
I'm also commenting on this.
I'm also commenting on this.
That is already known.
The Python interpreter will also disregard the following triple-quoted text, which may be used as a multiline comment:
This remark spans many lines.
Making Use of Offsets
Blank lines, which consist entirely of whitespace and may or may not include

comments, are completely disregarded by Python.
When using an interactive interpreter, to end a multiline statement, you need to input an empty physical line.
Holding a User's Attention
In the line that follows, the program shows the prompt with the sentence "Press the enter key to exit" and waits for the user to do anything − #!/usr/bin/python raw input. ("\n\nPress the enter key to exit.")
In this case, two additional lines are created using "\n\n" before the real line is shown. As soon as the key is pressed, the software terminates. One useful tip is to just leave the terminal window open until the user exits the program.
Consolidating Several Claims into One Line
As long as neither statement initiates a new code block, the semicolon (;) permits many statements on a single line. A snippet using the semicolon is seen here.
bring in sys; x = 'foo'; write (x + '\n') to sys.stdout
Collections of Statements as Suites
Suites are a way of organizing Python code into blocks that consist of individual statements. A header line and a suite are necessary for compound or complicated statements like if, while, def, and class.
One or more lines that compose the suite follow the header line, which starts the statement (with the keyword) and ends with a colon (:). As an example, what if the phrase is:
If not elifexpression, then suite

Command Line Arguments

You can get some basic information on how to operate many apps by running them. You may do this with Python by using the following command: $ python -h usage: python [option]... -c command | -m mod | file | -] Oh no!...

Contextual factors, choices, and arguments:
The command line interface is terminated by passing the program as a string.
-d: prevent parser output from being debugged (also use PYTHONDEBUG=x)
To disregard environment variables like PYTHONPATH, use the -E option.
Exit after printing this help message (-h).
Another alternative is to code your script so that it may take many choices. After you have mastered the fundamentals of Python, you should move on to more complex topics like command line arguments.
Lists in Python
One of Python's most flexible datatypes, the list may be expressed as an array of values (items) enclosed in square brackets and separated by commas. The fact that list elements do not always have to be of the same type is a crucial aspect of lists.
Putting items separated by commas inside square brackets is the simplest way to create a list. As an example, consider the following: list1 = ['physics', 'chemistry', 1997, 2000] While list2 has the numbers 1, 2, 3, 4, and 5, list3 contains the letters "a", "b", "c", and "d".
You can slice and concatenate lists, and their indices start at 0, just like string indices.
Python objects are immutable and may be organized into tuples. Similar to lists, tuples are sequences. The two main distinctions between lists and tuples are that lists may be modified whereas tuples cannot, and that tuples utilize parentheses instead of square brackets.

All it takes to create a tuple is to insert values separated by commas. You also have the option of putting these values, which are separated by commas, within parenthesis. As an illustration, consider the following example: tup1 = ('physics', 'chemistry', 1997, 2000); tup2 = (1, 2, 3, 4, 5); tup3 = "a", "b", "c", "d";
Two parenthesis holding nothing represent the empty tuple − tup1 = ();
Even if there is just one value in tup1 = (50,);, a comma is still required to create tuples with one value.
You may slice and concatenate tuple indices much like string indices, which start at 0.
Valuing Access in Tuples
Tibble values may be accessed by slicing them using square brackets and then using the index or indices to get the value at that index. Take the following example: − Live Demo #!/usr/bin/python
print "tup1[0]: ", tup1[0]; print "tup2[1:5]: ", tup2[1:5];" tup1 = ('physics', 'chemistry', 1997, 2000); tup2 = (1, 2, 3, 4, 5, 6, 7);;
The output of running the code above is tup1[0]: physics and tup2[1:5]: [2, 3, 4, 5].
Refreshing Tables
Getting at Dictionary Values
Use the well-known square brackets with the key to access dictionary components and get their value. This is an easy example —
Here is the live demo code: #!/usr/bin/python dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
"dict['Name']: ", dict['Name'] should be printed.
The output will be "dict['Age']: ", followed by the value of date.
The following outcome is generated when the code mentioned above is run →
key value in dictionary "Name": The Zara
key: "age"
An error will be shown if we try to access a data item using a key that is not in the dictionary.

Here is the live demo code: #!/usr/bin/python dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
output: print "dict['Alice']: ", dict['Alice']
The following outcome is generated when the code mentioned above is run →
phrase "Alice":
The most recent call's traceback:
Line 4 of the file "test.py" inside the
key error: 'Alice'; print "dict['Alice']: ", dict['Alice'];
Revising the Lexicon
As seen in the simple example below, you have the option to alter an existing entry, add a new entry or key-value combination, or remove an existing element from a dictionary.
Here is the live demo code: #!/usr/bin/python dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
revise current item by setting dict['Age'] to 8
Insert new item into the dictionary at "DPS School"
The output will be "dict['Age']: ", followed by the value of date.
The output will be "dict['School']: " followed by the contents of it.
The output is dict['Age']: 8 when the code in question is run.
the DPS School dictionary
Eliminate Vocabulary Terms

There are two options for clearing a dictionary: removing certain items or clearing it entirely. Deleting a whole dictionary at once is also an option.
Just use the del command to erase a whole dictionary directly. This is an easy example —
Here is the live demo code: #!/usr/bin/python dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
delete all items from dict del dict; # delete entry with key 'Name' from dict.clear(); ## remove all dictionary entries
The output will be "dict['Age']: ", followed by the value of date.
The output will be "dict['School']: " followed by the contents of it.
The outcome is as follows. The fact that the dictionary is no longer available after del dict is the reason an exception is triggered. dict['Age'] is being referred to.
The most recent call's traceback:
Line 8 of file "test.py" inside the
Output: "dict['Age']: ", dict['Age'];
Unsubscriptable object of type 'type': TypeError
In the section that follows, the del() function is covered.

Properties of Dictionary Keys

Dictionary values have no restrictions. They can be any arbitrary Python object, either standard objects or user-defined objects. However, same is not true for the keys.

There are two important points to remember about dictionary keys −

(a) More than one entry per key not allowed. Which means no duplicate key is allowed. When duplicate keys encountered during assignment, the last assignment wins. For example −

Live Demo

```
#!/usr/bin/python

dict = {'Name': 'Zara', 'Age': 7, 'Name': 'Manni'}

print "dict['Name']: ", dict['Name']
```

When the above code is executed, it produces the following result −

```
dict['Name']:  Manni
```

(b) Keys must be immutable. Which means you can use strings, numbers or tuples as dictionary keys but something like ['key'] is not allowed. Following is a simple example –

Live Demo

```
#!/usr/bin/python

dict = {['Name']: 'Zara', 'Age': 7}

print "dict['Name']: ", dict['Name']
```

When the above code is executed, it produces the following result −

```
Traceback (most recent call last):

  File "test.py", line 3, in <module>

dict = {['Name']: 'Zara', 'Age': 7};

TypeError: unhashable type: 'list'
```

Tuples are immutable which means you cannot update or change the values of tuple elements. You are able to take portions of existing tuples to create new tuples as the following example demonstrates −

Live Demo

```
#!/usr/bin/python

tup1 = (12, 34.56);

tup2 = ('abc', 'xyz');

# Following action is not valid for tuples

# tup1[0] = 100;

# So let's create a new tuple as follows

tup3 = tup1 + tup2;

print tup3;
```

When the above code is executed, it produces the following result −

```
(12, 34.56, 'abc', 'xyz')
```

Delete Tuple Elements

Removing individual tuple elements is not possible. There is, of course, nothing wrong with putting together another tuple with the undesired elements discarded.

To explicitly remove an entire tuple, just use the del statement. For example −

Live Demo

```
#!/usr/bin/python

tup = ('physics', 'chemistry', 1997, 2000);

print tup;

del tup;

print "After deleting tup : ";

print tup;
```

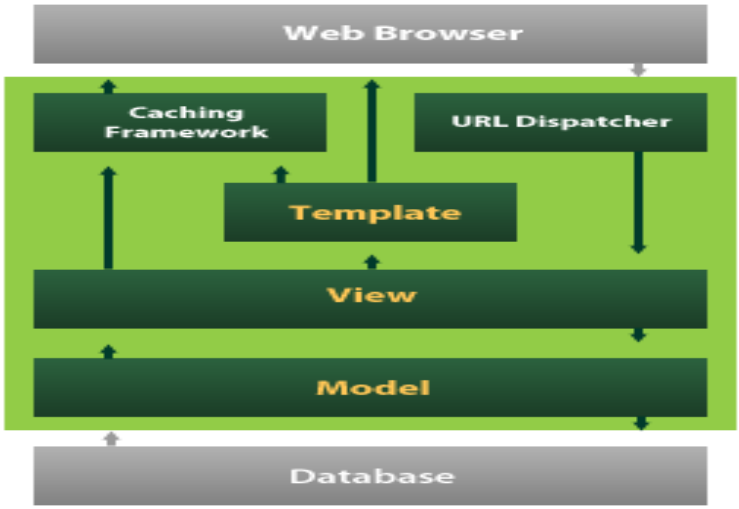
This produces the following result. Note an exception raised, this is because after del tup tuple does not exist any more −

```
('physics', 'chemistry', 1997, 2000)

After deleting tup :

Traceback (most recent call last):

  File "test.py", line 9, in <module>

    print tup;

NameError: name 'tup' is not defined
```

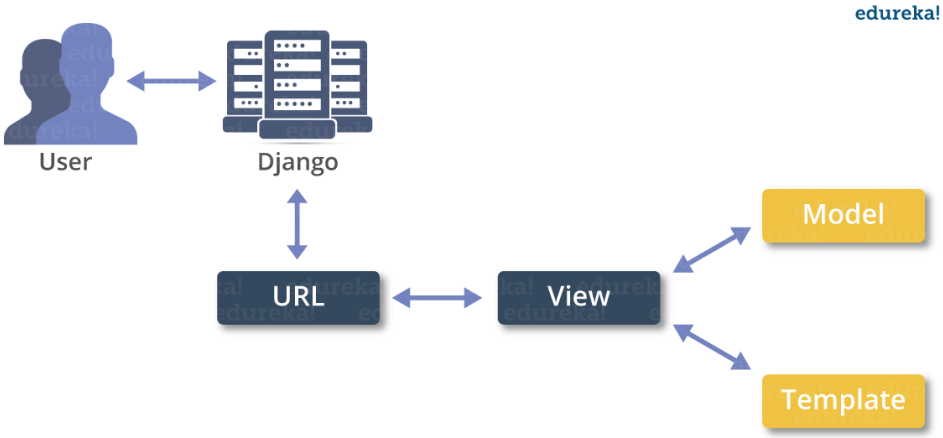
4.3 DJANGO

Quick development and simple, practical design are encouraged by the Django web framework, which is a high-level Python framework. Crafted by seasoned programmers, it handles most of the tedious aspects of web development, allowing you to concentrate on creating your app without having to start from scratch. Plus, it's free and open source.

Django's primary goal is to ease the creation of complex, database-driven websites. Django emphasizes reusabilityand "pluggability" of components, rapid development, and the principle of don't repeat yourself. Python is used throughout, even for settings files and data models.



Django also provides an optional administrative create, read, update and delete interface that is generated dynamically through introspection and configured via admin models.



Create a Project

Just open a terminal or command prompt on your Windows or Linux machine, go to the location where you want your project to be created, and paste this code there.
"$ django-admin startprojectmyproject"
A "myproject" folder will be created with the following structure.

the file located at myproject/manage.py
init.py, settings.py, urls.py, and myproject/

Filename: wsgi.py
The Framework of the Project
The "myproject" folder only serves as a container for your project and includes two components: manage.py and In order to launch the development server, sync databases, and interact with your project via command line, you may think of this file as your project's local django-admin. If you want to see all the commands available in manage.py, you may use the following command: $ python manage.py help.
The folder named "myproject" → You will find your project's real Python package in this subdirectory. There are four files in it.
The file __init__.py is generated. Please consider this folder as a package only for Python.
setup.py → The file urls.py contains all the URLs to your project and the function to call, as the name suggests, your project settings. An outline of your project's main points.
*/ wsgi.py In the event that WSGI deployment of your app is required.
Before You Begin Your Project
Go to the subdirectory myproject/settings.py to find your project's configuration file. Here are a few crucial settings that you may have to adjust .
LOG = True
Turn on or off debugging for your project by toggling the switch. Switching to debug mode could help you understand the issue with your project. In the middle of a live project, setting it to "True" is a recipe for catastrophe. For static file serving to work with the Django light server, make sure this is set to "True" in the configuration. Remain in development mode at all times.
The data sets are initialized with the following parameters: 'ENGINE': 'django.db.backends.sqlite3', 'NAME': 'database.sql', 'USER': '', 'PASSWORD': '', 'HOST': '', 'PORT': '', and so on.
The sentence is already paraphrased. For the SQLite engine, see the preceding example. Django is compatible with many database systems, including MySQL, PostgreSQL, Oracle, and NoSQL DB, as mentioned above.
Django MongoDB Engine (MongoDB)
Verify that you have the appropriate database driver installed before configuring any new engine.
Additional settings such as TIME_ZONE, LANGUAGE_CODE, TEMPLATE, etc., may also be configured.
The next step is to test the functionality of your newly created and configured project.
"$ manage.py runserver" in Python
When you execute the code provided above, you should get an output similar to this →
Model validation...
Found zero errors.
At 11:41:50 on September 3, 2015
The settings'myproject.settings' are being used with Django version 1.6.11.
The development server may be started at http://127.0.0.1:8000/.
Use the Control key to exit the server.
Multiple apps come together to form a project. An example of an application would be a website's contact form; it serves a purpose and may be used for other projects. Imagine it as a part of your project's module.
Make a Program
Presumably, you have landed at the project folder. Within our primary "myproject"

folder, go to the manage.py file.

run "appmyapp" in Python
Django automatically creates a "myapp" folder with the application structure − myapp/ __init__.py once you create an application, much like a project.
This folder should be seen as a package by Python, so here are some files to ensure it does: admin.py, models.py, tests.py, views.py, and __init__.py.
To make the app customizable on the admin interface, you may use the admin.py file.
All of the application models are saved in the models.py file.
The location of your unit tests is tests.py.
Your application views are located in views.py.
Make Your Application Known to the Project
We now have an application called "myapp" and need to link it to our Django project called "myproject" for registration. In order to accomplish this, modify the INSTALLED_APPS tuple in your project's settings.py file by adding the name of your app: INSTALLED_APPS = ('django.contrib.admin', 'django.contrib.auth', 'django.contrib.contenttypes', 'django.contrib.sessions', 'django.contrib.messages', 'django.contrib.staticfiles','myapp',)
creating a model is quite similar to creating a form in Django. Inheriting from the Django class is all that's required here; the form fields will be the attributes of this class. To house our app's forms, let's create a files.py file in the myapp subdirectory. A login form will be built.
code: utf-8 #import forms from django into myapp/forms.py

User is assigned the value of forms by the LoginForm class. The maximum length of the CharField is 100.
forms are passwords. The code uses a password input widget and a CharField.

We want to hide the password, not show it, therefore we may use the "widget" parameter that the field type can accept for HTML rendering. Django has a plethora of different widgets: For dates, checkboxes, etc., use DateInput.
Displaying Data using Form
Two types of queries are supported by HTTP: GET and POST. All data given via POST may be accessible via the request, which is a property named "method" in the Django request object that is passed as an argument to your view. The dictionary that is used for posting.
Our myapp/views.py file needs a login view, so let's make one. #-*- coding: utf-8 -*- the forms in myapp the import statement for LoginForm defines the login method with the request parameter:
the request triggers the "not logged in" status for the username. parameter is set to "POST":
If the form is legitimate, then set the username to the cleaned data. Otherwise, get the submitted form by using LoginForm(request.POST). This is the username column. return render(request, 'loggedin.html', {"username": username}) if not: MyLoginForm = Loginform()
The loggedin.html will be used to publish the results of the login form to the view. We need the login form template in order to do the testing. It will be called login.html.
The following code snippet is being executed:

"Csrf token"
The following code snippet displays a password input with the following parameters:

<=="" center="">

The sentence says: Join in It is not a paraphrase.
Click the button.

A login form will be shown by the template, and the outcome will be sent to the login view up top. The element in the template is there only to protect your site against Cross-site Request Forgery (CSRF) attacks, which you may have seen.
"Csrf token"
After the form is processed, we will require the loggedin.html template, which we will get after the login template is ready.
the HTML code for the page and its body
The person you are is: {{logging in}} It is not a paraphrase.
this is the body tag

All we need now are two URLs to get going: use the file myapp/urls.py in the django.conf.urls module reference the django.views.generic package and import its patterns and URLs 'myapp.views',
url(r'^connection/',TemplateView.as_view(template_name = 'login.html')), and
url(r'^login/', 'login', name = 'login')) make up the urlpatterns.
The login.html template is shown when we go to "/myapp/connection".
Arranging Meetings
To enable sessions in Django, go to your project settings. Python, by inserting a few lines into the INSTALLED_APPS and MIDDLEWARE_CLASSES configuration parameters. Adding 'django.contrib.sessions.middleware.SessionMiddleware' to MIDDLEWARE_CLASSES is something that should be done when building the project, but knowing it's a nice idea.
Additionally, 'django.contrib.sessions' need to be present in INSTALLED_APPS.
You have the option to customize the engine to store information in several methods, such as a file or cache, however by default, Django stores session information in the database (django_session table or collection).
With session enabled, a session (dict) property is appended to every request, which is the first parameter of every Django view.
Let's make a little example to learn about session creation and saving. Refer to the Django form processing and Django cookies handling chapters for our previous work on a basic login system. We may prevent you from seeing the login form when you visit our login page if you allow us to save your username in a cookie. Simply said, by storing cookies on the server, we can improve the security of our login mechanism employed in Django's cookies handling.

To begin, let's modify our login page such that our username is saved in a cookie on the server side. def login(request): username = 'not logged in' if request.method When the request type is 'POST', the following code will run: MyLoginForm = LoginForm(request.POST) If the MyLoginForm is valid, then the cleansed data will be assigned to the username variable. This is the username column.
session of the request "username" is now "username"
Otherwise, set MyLoginForm to LoginForm().
The code is returning the 'loggedin.html' page with the "username" parameter set to the actual username.
Our login form will not be shown if the cookie is set. Here's the code: ``def formView(request): if request.session.has_key('username'): username = request.session['username'] ``
execute the code within the request and return the 'loggedin.html' page with the "username" as the parameter.
"else": return the rendered version of the requested page ('login.html')
Now we can modify the url.py file to align it with our new view by importing patterns and url from django.conf.urls and django.views.generic, respectively. The following code snippet is creating a template view with url patterns:'myapp.views', url(r'^connection/','formView', name = 'loginform'), and url(r'^login/', 'login', name = 'login').
The following page will be shown when you enter /myapp/connection.

4.4 SOURCE CODE

VIEWS.PY

```
from ast import alias

from concurrent.futures import process

from django.shortcuts import render

# Create your views here.

from django.shortcuts import render, HttpResponse

from django.contrib import messages

import Alzheimers_Disease_Prediction

from .forms import UserRegistrationForm

from .models import UserRegistrationModel

from django.conf import settings

import os

def UserRegisterActions(request):

if request.method == 'POST':
```

```
form = UserRegistrationForm(request.POST)
if form.is_valid():
print('Data is Valid')
form.save()
messages.success(request, 'You have been successfully registered')
form = UserRegistrationForm()
return render(request, 'UserRegistrations.html', {'form': form})
else:
messages.success(request, 'Email or Mobile Already Existed')
print("Invalid form")
else:
form = UserRegistrationForm()
return render(request, 'UserRegistrations.html', {'form': form})

def UserLoginCheck(request):
if request.method == "POST":
loginid = request.POST.get('loginid')
pswd = request.POST.get('pswd')
print("Login ID = ", loginid, ' Password = ', pswd)
try:
check = UserRegistrationModel.objects.get(
loginid=loginid, password=pswd)
status = check.status
print('Status is = ', status)
if status == "activated":
request.session['id'] = check.id
request.session['loggeduser'] = check.name
request.session['loginid'] = loginid
request.session['email'] = check.email
print("User id At", check.id, status)
```

```
return render(request, 'users/UserHomePage.html', {})
else:
messages.success(request, 'Your Account Not at activated')
return render(request, 'UserLogin.html')
except Exception as e:
print('Exception is ', str(e))
pass
messages.success(request, 'Invalid Login id and password')
return render(request, 'UserLogin.html', {})
def UserHome(request):
return render(request, 'users/UserHomePage.html', {})
def DatasetView(request):
path = settings.MEDIA_ROOT + "//" + 'alzheimers_prediction_dataset.csv'
df = pd.read_csv(path)
df = df.to_html
return render(request, 'users/viewdataset.html', {'data': df})
import pandas as pd
df = pd.read_csv(r'C:\Users\lenovo\Desktop\nandini\Alzheimers_Disease_Prediction\media\alzheimers_prediction_dataset.csv')
x = df.drop('Alzheimer_Diagnosis', axis=1)
y=df['Alzheimer_Diagnosis']
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report ,confusion_matrix
import matplotlib.pyplot as plt
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score, log_loss, roc_curve, auc
from sklearn.linear_model import LogisticRegression
import numpy as np
import seaborn as sns
path = r'balanced_data.csv'
```

```
df = pd.read_csv(path)
df.fillna(0, inplace=True)
x_train, x_test, y_train, y_test = train_test_split(x,y,test_size=0.2,random_state=42)
model = LogisticRegression(random_state=42)
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
# Create your views here.
from django.shortcuts import render
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix,
log_loss, roc_curve, auc
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, train_test_split
import matplotlib.pyplot as plt
import seaborn as sns
def Training(request):
# Define model at the start of the function
model = LogisticRegression(max_iter=1000, random_state=42)
# Train/test split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
# Fit the model on the training data
model.fit(x_train, y_train)
predictions = model.predict(x_test)
accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {accuracy:.2f}")
# Make predictions on the test data
y_pred = model.predict(x_test)
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
# Confusion matrix plot
plt.figure(figsize=(6, 4))
```

```
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
xticklabels=['No Alzheimer\'s', 'Alzheimer\'s'],
yticklabels=['No Alzheimer\'s', 'Alzheimer\'s'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()
# Train/test accuracies
y_train_pred = model.predict(x_train)
y_test_pred = model.predict(x_test)
train_acc = accuracy_score(y_train, y_train_pred)
test_acc = accuracy_score(y_test, y_test_pred)
print(f"Train Accuracy: {train_acc * 100:.2f}%")
print(f"Test Accuracy: {test_acc * 100:.2f}%")
# Cross-validation
train_accuracies = []
val_accuracies = []
train_losses = []
val_losses = []
kf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for train_index, val_index in kf.split(x, y):
x_train_fold, x_val_fold = x.iloc[train_index], x.iloc[val_index]
y_train_fold, y_val_fold = y.iloc[train_index], y.iloc[val_index]
# Train the model on each fold
model.fit(x_train_fold, y_train_fold)
# Train and validation metrics
y_train_pred_fold = model.predict(x_train_fold)
y_train_prob_fold = model.predict_proba(x_train_fold)
train_accuracies.append(accuracy_score(y_train_fold, y_train_pred_fold))
train_losses.append(log_loss(y_train_fold, y_train_prob_fold))
```

```
y_val_pred_fold = model.predict(x_val_fold)
y_val_prob_fold = model.predict_proba(x_val_fold)
val_accuracies.append(accuracy_score(y_val_fold, y_val_pred_fold))
val_losses.append(log_loss(y_val_fold, y_val_prob_fold))
# Plot Training vs Validation Accuracy
plt.figure(figsize=(14, 5))
plt.subplot(1, 2, 1)
plt.plot(range(1, len(train_accuracies) + 1), train_accuracies, label='Train Accuracy',
marker='o')
plt.plot(range(1, len(val_accuracies) + 1), val_accuracies, label='Validation Accuracy',
marker='o')
plt.xlabel('Fold')
plt.ylabel('Accuracy')
plt.legend()
plt.title('Training vs Validation Accuracy')
# Plot Training vs Validation Loss
plt.subplot(1, 2, 2)
plt.plot(range(1, len(train_losses) + 1), train_losses, label='Train Loss', marker='o')
plt.plot(range(1, len(val_losses) + 1), val_losses, label='Validation Loss', marker='o')
    plt.xlabel('Fold')
    plt.ylabel('Log Loss')
    plt.legend()
    plt.title('Training vs Validation Loss')
    plt.tight_layout()
    plt.show()
    # ROC curve and AUC
    y_test_prob = model.predict_proba(x_test)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, y_test_prob)
    roc_auc = auc(fpr, tpr)
    plt.figure(figsize=(8, 6))
```

```
        plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {roc_auc:.2f})')
    plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title('Receiver Operating Characteristic (ROC) Curve')
    plt.legend(loc="lower right")
    plt.show()
   # Return context to template
    context = {
        'accuracy':accuracy,
        'train_acc': train_acc,
        'test_acc': test_acc
    }
    return render(request, 'users/training.html', context)
def Prediction(request):
    if request.method == "POST":
        ag = request.POST.get('age')
        cl = request.POST.get('cholesterol_levels')
        cd = request.POST.get('chest_discomfort')
        ohf1 = request.POST.get('other_health_factor_1')
        ohf2 = request.POST.get('other_health_factor_2')
        # ag = int(ag) if ag else 0
        # cl = int(cl) if cl else 0
        cd = 1 if cd == 'Yes' else 0
        # ohf1 = float(ohf1) if ohf1 else 0
        # ohf2 = float(ohf2) if ohf2 else 0
        input_df = pd.DataFrame({
            'Age': [ag],
            'Cholesterol_Levels': [cl],
            'Chest_Discomfort': [cd],
```

```
        'Other_Health_Factor_1': [ohf1],
        'Other_Health_Factor_2': [ohf2]
    })
    print(input_df)
    print("Input DataFrame before encoding:", input_df)
    op = model.predict(input_df)
    prediction_label = "Patient contain Alzheimer's" if int(op[0]) == 1 else "Patient does not contain Alzheimer's"
    print("Prediction label:", prediction_label)
    context = {
    'prediction': prediction_label
    }
    return render(request, 'users/predict_form.html', context)
  return render(request, 'users/predict_form.html')
```

BASE.HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Alzheimer's Disease Prediction Using Modern Machine Learning Techniques</title>
  <!-- Bootstrap CSS -->
  <link href="https://maxcdn.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
  <!-- Navbar -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <a class="navbar-brand" href="#">Alzheimer's Disease Prediction Using Modern Machine Learning Techniques</a>
```

```
<button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
<span class="navbar-toggler-icon"></span>
</button>
<div class="collapse navbar-collapse" id="navbarNav">
<ul class="navbar-nav ml-auto">
<li class="nav-item">
<a class="nav-link" href="{% url 'index' %}">Home</a>
</li>
<li class="nav-item">
<a class="nav-link" href="{% url  'UserLogin' %}">User</a>
</li>
<li class="nav-item">
<a class="nav-link" href="{% url 'AdminLogin' %}">Admin</a>
</li>
<li class="nav-item">
<a class="nav-link" href="{% url 'UserRegister' %}">Register</a>
</li>
</ul>
</div>
</nav>

{% block contents %}
{% endblock %}

<!-- Bootstrap JS and dependencies -->
<script src="https://code.jquery.com/jquery-3.5.1.slim.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/@popperjs/core@2.6.0/dist/umd/popper.min.js"></script>
```

```
<script
src="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/js/bootstrap.min.js"></script
>

</body>

</html>
```

5. SYSTEM TESTING

Finding mistakes is the purpose of doing tests. Testing aims to identify vulnerabilities in a service or product that might compromise its security. Parts, subassemblies, and the final product may all be tested using this technique. Software is tested to make sure it doesn't crash or do anything else that users may not like. There is a wide variety of tests available. Different testing needs are met by developing a range of tests.

The 5.1 Unit Test

Programs are subjected to unit testing in order to ensure that their fundamental logic is operating as intended and that program inputs generate legitimate outputs. Verify all decision branches and the internal code flow by yourself. At this stage, we check that certain parts of the application's code are working as they should. The tasks are completed one by one before they are merged. What comes out of this intrusive structural study will depend on how well we understand its structure. Testing the operation of specific parts inside a larger program, system setup, or business process is what unit testing is all about. The purpose of unit testing is to verify that all business process routes adhere to the defined criteria and have well-defined inputs and outputs.

5.1.1 Test Objectives

Every field must be operational for you to proceed.

In order to access the pages, you must click on the given link.

Response, message, or input screens must be instantaneous.

5.1.2 Checklist Items

• Verify that the entries are properly formatted.

• Making sure there are no duplicate entries is crucial.

By clicking on any link, you should be able to visit the correct website.

5.2 Integrity Testing

To ensure that many software applications can work together as expected is what an integration test is all about. The main emphasis of event-driven testing is on the output on the screen or in the field. Integration tests ensured accuracy and consistency throughout, whereas unit tests demonstrated that individual components were adequate. The primary purpose of integration testing is to identify problems that arise when components are combined.

Fifthly, System Testing

System testing is a must for any integrated software system to guarantee proper operation. It verifies a configuration to deliver outcomes that are comprehensible and predictable. The configuration-oriented system integration test demonstrates system testing. Integration points and pre-driven process connections are the primary targets of system testing, which is based on documented methods.

Testing on Empty Spaces 5.4

The software tester doing white box testing has prior knowledge of the program's functionality, purpose, language, and structure. Its objective is that. When used in conjunction with a black box level, it completes the testing process.

5.5 Investigating the Mystery

In black box testing, the testers do not have access to the program's source code, architecture, or internal workings. such most other types of tests, black box tests need a final source document, such a requirements or specification document, to be created. During a black box test, it is recommended to minimize user contact while evaluating software. Take a peek inside; there's nothing visible. Without taking the program's internals into account, the test just reads data and prints the results.
5.6 Insuring Succes
Users' Compliance Involvement of the end user is crucial throughout the testing phase of any project. It also ensures the system performs as anticipated. Results from the Examination: Every one of the test cases that were stated before ran without a hitch. Nothing was wrong with it.

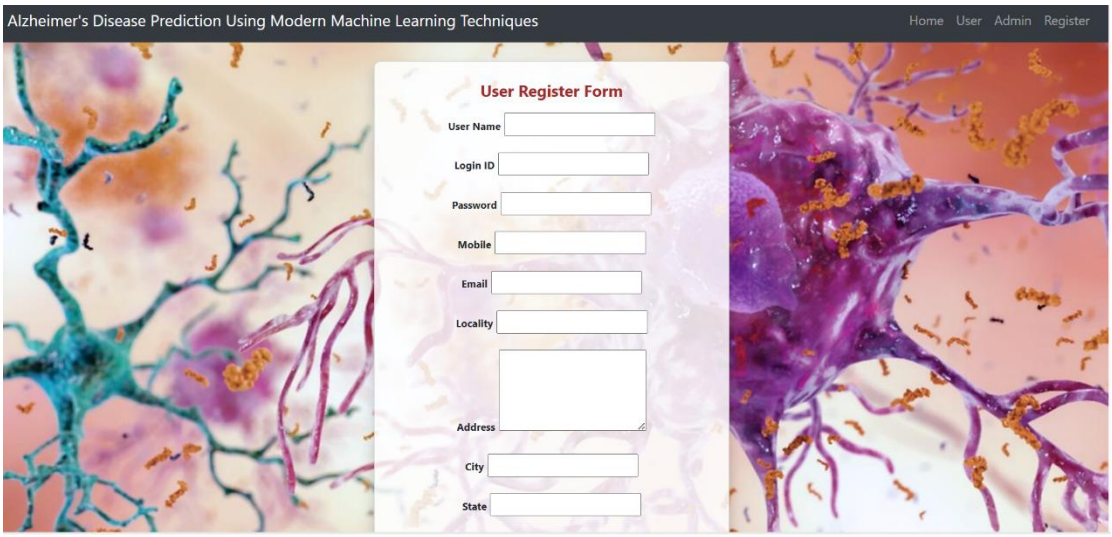
5.7 Sample Test Cases

S.no	Test Case	Excepted Result	Result	Remarks(IF Fails)
1	User Register	If User registration successfully.	Pass	If already user email exist then it fails.
2	User Login	If Username and password is correct then it will getting valid page.	Pass	Un Register Users will not logged in.
3	Dataset view	Used data set data will be displayed	Pass	Results not true failed
4	Prediction	Displays the type of cyber attack attacked to the client or user	Pass	If Results is not true failed
5	Admin login	Admin can login with his login credential. If success he get his home page	Pass	Invalid login details will not allowed here
6	Admin can activate the register users	Admin can activate the register user id	Pass	If user id not found then it won't login.

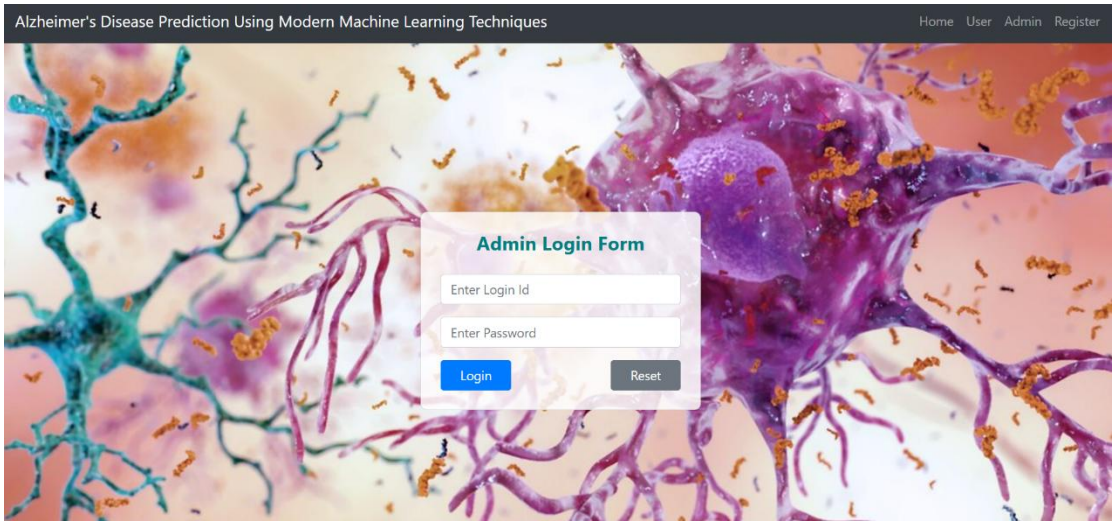
Table 5.1: Sample Test Case

6.OUTPUTSCREENS

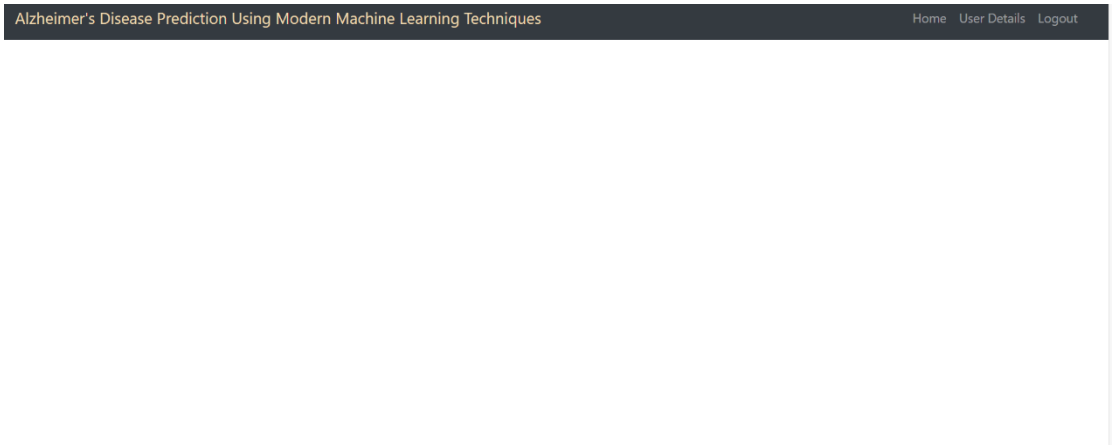
Output Screen 6.1: HOME PAGE



Output Screen 6.2: REGISTER PAGE

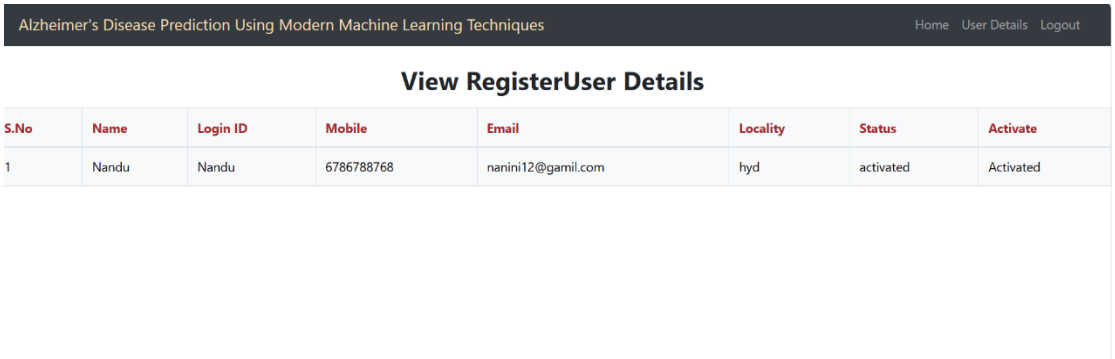


Output Screen 6.3: ADMIN LOGIN PAGE



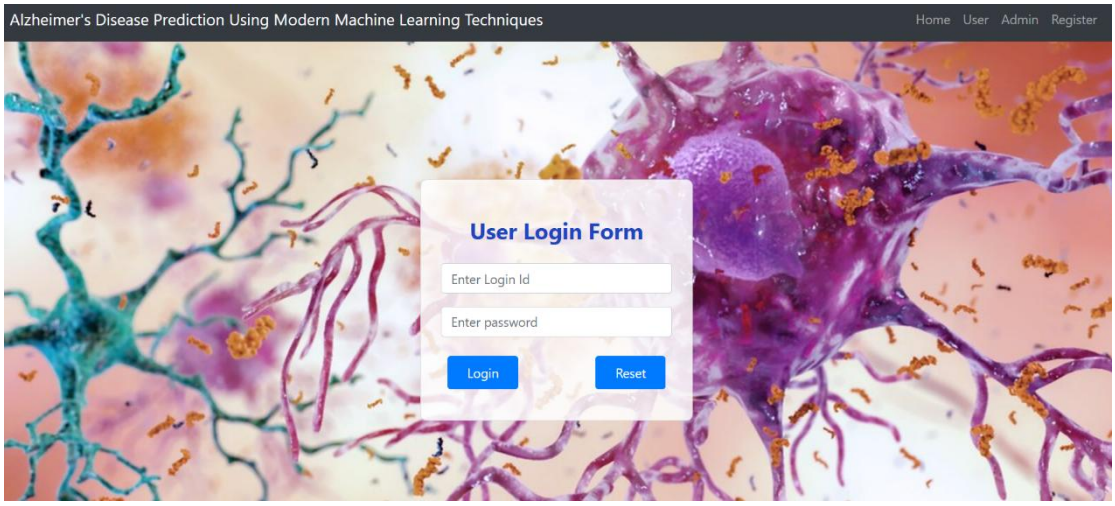
Output Screen 6.4: ADMIN HOME PAGE

USER DETAILS PAGE

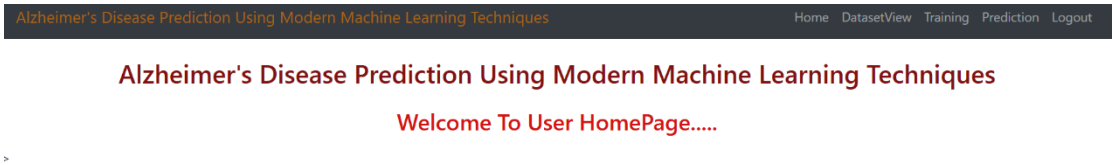


S.No	Name	Login ID	Mobile	Email	Locality	Status	Activate
1	Nandu	Nandu	6786788768	nanini12@gamil.com	hyd	activated	Activated

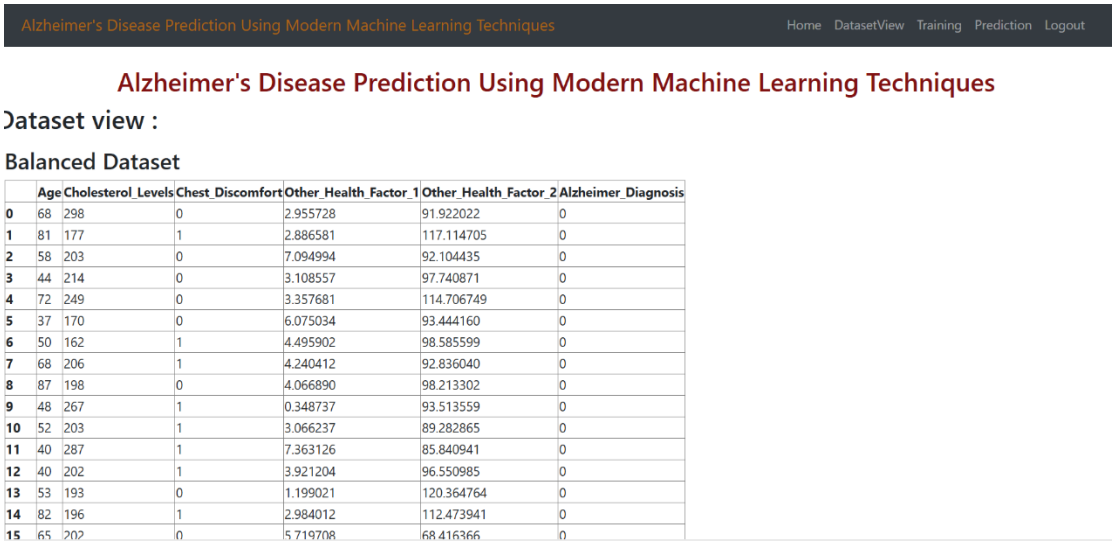
Output Screen 6.5: USER DETAILS PAGE



Output Screen 6.6: USER LOGIN PAGE

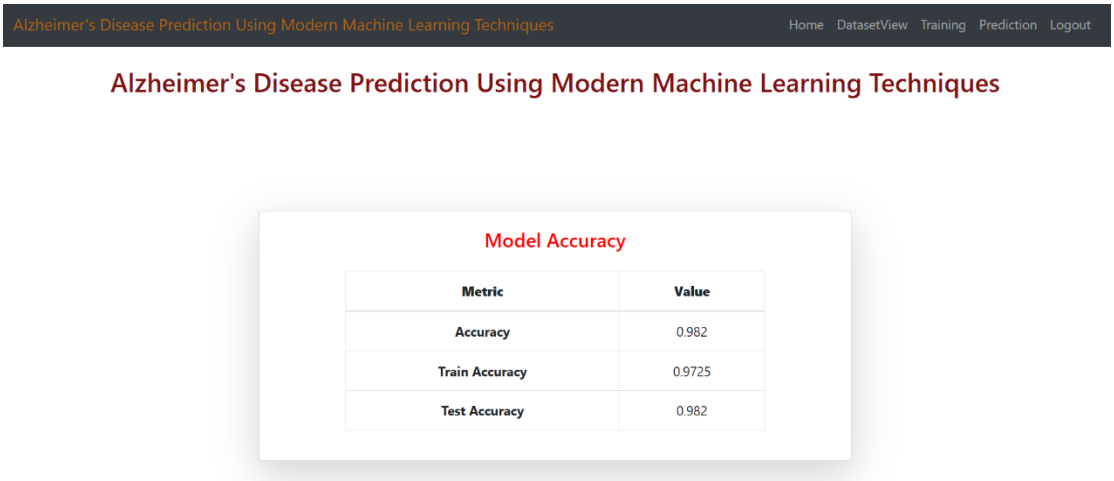


Output Screen 6.7: USER HOME PAGE

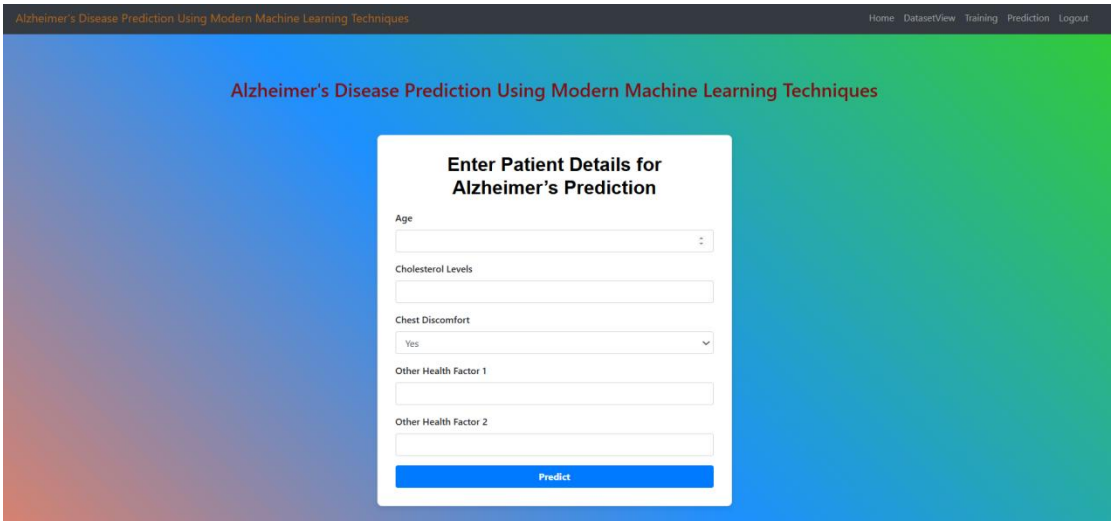


	Age	Cholesterol_Levels	Chest_Discomfort	Other_Health_Factor_1	Other_Health_Factor_2	Alzheimer_Diagnosis
0	68	298	0	2.955728	91.922022	0
1	81	177	1	2.886581	117.114705	0
2	58	203	0	7.094994	92.104435	0
3	44	214	0	3.108557	97.740871	0
4	72	249	0	3.357681	114.706749	0
5	37	170	0	6.075034	93.444160	0
6	50	162	1	4.495902	98.585599	0
7	68	206	1	4.240412	92.836040	0
8	87	198	0	4.066890	98.213302	0
9	48	267	1	0.348737	93.513559	0
10	52	203	1	3.066237	89.282865	0
11	40	287	1	7.363126	85.840941	0
12	40	202	1	3.921204	96.550985	0
13	53	193	0	1.199021	120.364764	0
14	82	196	1	2.984012	112.473941	0
15	65	202	0	5.719708	68.416366	0

Output Screen 6.8: DATASET



Output Screen 6.8: TRAINING PAGE



Output Screen 6.9: PREDICTION PAGE

7. CONCLUSION

Machine learning algorithms can reliably predict when Alzheimer's disease may start, according to this study's findings. Logistic Regression, Random Forest Classifier, K-Nearest Neighbors, and Artificial Neural Networks were evaluated for their ability to predict the beginning of Alzheimer's disease using a dataset that included demographic and clinical information. Based on our findings, all four algorithms performed well when it came to predicting the onset of disease. Logistic Regression outperformed all of the other algorithms with a 94% accuracy rate, while Random Forest Classifier came in at a little lower 90%. Nevertheless, K-Nearest Neighbors and Artificial Neural Networks performed well, achieving 88% accuracy. These results demonstrate the potential of machine learning algorithms to forecast the onset of Alzheimer's disease, which could lead to earlier diagnosis and treatment. Caregivers are relieved and patients and their family enjoy a far higher quality of life when Alzheimer's disease is detected early. As shown by the remarkable precision of the algorithms evaluated in this study, it is essential to use machine learning to evaluate complex medical data in order to improve clinical decision-making. We need to dig further into these algorithms' potential to enhance the precision of Alzheimer's disease diagnosis and therapy.

8.FUTURE ENHANCEMENT

A number of machine learning techniques, such as Artificial Neural Networks, K-Nearest Neighbor, Logistic Regression, and Random Forest, show significant potential as Alzheimer's disease detectors. Machine learning algorithms can learn more complex patterns and make better predictions when more data is available to them. This will allow for early diagnosis of Alzheimer's disease and thus improved treatment for sufferers.

Using machine learning, personalized treatment plans for Alzheimer's patients may be created. Algorithms based on machine learning may evaluate patient data to determine which medications are most beneficial for various types of patients, allowing for more efficient and successful treatments. Additionally, machine learning has the potential to reveal variables that increase the likelihood of developing Alzheimer's disease. Machine learning algorithms can learn more complex patterns and make better predictions when more data is available to them. This will allow for early diagnosis of Alzheimer's disease and thus improved treatment for sufferers.

9. BIBLIOGRAPHY

REFERENCES

According to an article published in the International Journal of Scientific Research in Engineering and Management (IJSREM) in 2021, Gomathy and Naidu discuss the use of machine learning techniques for disease prediction.

"Disease Prediction Using Machine Learning Techniques" (IEEE, 2021, pp. 962–966) by Mallela, C. R., Bhavani, L. R., and Ankayarkanni, B.
"A Machine Learning Approach to Prevent Cancer" (pp.112-141, 2021) by Chatterjee and Roy. IGI Global.

In 2013, the authors Tarigoppula, Rao, and Narayan published an article titled "Intelligent Parkinson Disease Prediction Using Machine Learning Algorithms" in the International Journal of Engineering and Information Technology, volume 3, pages 212-215.

[In the 2020 issue of the ICTACT Journal of Soft Computing, Benjamin examined the use of ensemble learning algorithms to improve the accuracy of heart disease prediction (vol. 10, pp. 2084–2089).

In 2020, Wang, Chakraborty, and Chakraborty published an article titled "Predicting the risk of chronic kidney disease (ckd) using machine learning algorithm" in the MDPI Applied Sciences journal. The article can be found on page 202. "Applications of Machine Learning in Disease Prescreening," by Kumar, U., 2018, pp. 1052–1084.

The article "Disease Prediction Using Machine Learning" was published in the International Journal of Engineering and Technology (IRJET) in 2019.

An article titled "Disease Prediction From Various Symptoms Using Machine Learning" was published in 2020 by Keniya et al. and authored by Khakharia, Shah, Gada, Manjalkar, Thaker, Warang, and Mehendale.

The paper "Heart Disease Prediction and classification Using Machine Learning Algorithms Optimized by Particle Swarm Optimization and Ant Colony Optimization" was published in the International Journal of Engineering and Information Systems (IJIES) in 2019.

[11] "Effective heart disease prediction using hybrid machine learning techniques" (March 2019) in IEEE, volume 7, pages 81542–8554, by Mohan, Thirumalai, and Srivastava.